ESC392 Digital Electronics Lab

Part C: Real-World Application & Code-to-Circuit Assignments

Lab Assignment 6: Application Mapping of Sequential Blocks

1. Flip-Flop (1-bit Memory Cell) Application: Keypad Switch Debouncing

Real System: Industrial keypad entry systems or mechanical push-buttons.

Role Description: Mechanical switches "bounce" when pressed, creating multiple false HIGH/LOW signals in milliseconds. An SR (Set-Reset) Latch, constructed from two cross-coupled NAND gates, functions as a 1-bit memory cell to debounce the switch. When the mechanical contact first strikes the "Set" terminal, the latch output (Q) flips to HIGH and holds that stable value indefinitely, ignoring subsequent mechanical chatter. This ensures a clean, single digital pulse is delivered to the processor.

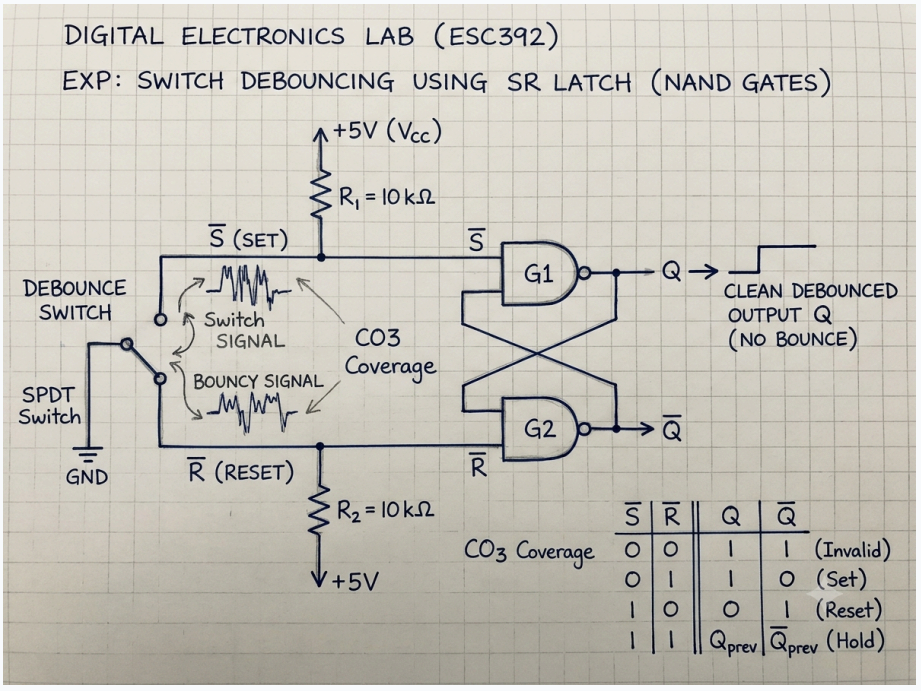


Figure 1: Hardware schematic for switch debouncing using an active-low SR Latch.

2. Mod-N Counter Application: 60-Second Timer in a Digital Clock

Real System: The seconds/minutes counting stage of a 24-hour digital clock system.

Role Description: A Mod-60 counter increments from 0 to 59 and resets precisely on the 60th pulse. Because 60 unique states are required, the architecture utilizes 6 cascaded Flip-Flops (providing $2^6 = 64$ total states).

Implementation Logic: The circuit is configured as a standard asynchronous up-counter. The binary representation of 60 is 111100 (Q_5, Q_4, Q_3, Q_2, Q_1, Q_0). To truncate the count at 60, a combinational 4-input AND gate is connected directly to outputs Q_5, Q_4, Q_3, and Q_2. When the count reaches 60, the AND gate evaluates to HIGH. This signal is routed to the active-high asynchronous CLEAR (CLR) pins of all 6 flip-flops, instantly resetting the entire register to 000000 and simultaneously propagating a carry pulse to the adjacent minutes counter.

3. Shift Register Application: SIPO in Serial-to-Parallel Communication

Real System: UART receivers decoding serial data streams from USB or external sensors.

Role Description: Microcontrollers process data efficiently in parallel chunks (e.g., 8-bit buses), but transmission lines send data sequentially (1 bit at a time) to minimize wiring infrastructure. A Serial-In Parallel-Out (SIPO) shift register bridges this gap.

Operation: Incoming data is fed into the D-input of the primary flip-flop. With every clock pulse, the stored bit is shifted to the subsequent flip-flop in the chain. After precisely 8 clock pulses, an entire byte is reconstructed across the 8 parallel output pins (Q_0 through Q_7), where it is instantly read by the system's parallel data bus.

> **CO Coverage (Lab 6):** CO3 (primary focus on sequential components), CO1 (reinforced logic application).

Lab Assignment 7: Design a Mini Automation System

System Selected: Digital Control Logic for an Automatic Parking Barrier.

1. Definition of Inputs, Outputs, and Constraints

- **Inputs:** A (Entrance IR Sensor: 1 = Car is waiting to enter), B (Clearance IR Sensor located under the barrier: 1 = Car is currently passing).
- **Outputs:** `MU` (Motor Up: 1 = Raise the barrier), `MD` (Motor Down: 1 = Lower the barrier).
- **Safety Constraint:** The barrier must remain elevated as long as B = 1, ensuring the gate never closes while a vehicle is actively passing underneath.

2. Finite State Machine (FSM) Derivation

The system is modeled with three primary states: S0 (Closed/Idle), S1 (Opening/Vehicle Waiting), and S2 (Vehicle Passing).

Present State (Q_1, Q_0)	Inputs (A, B)	Next State (D_1, D_0)	Motor Up (MU)	Motor Down (MD)
00 (S0)	0, 0	00 (S0)	0	1
00 (S0)	1, 0	01 (S1)	1	0
01 (S1)	X, 0	01 (S1)	1	0
01 (S1)	X, 1	10 (S2)	1	0
10 (S2)	X, 1	10 (S2)	1	0
10 (S2)	X, 0	00 (S0)	0	1

3. Edge Case Handling

Reversing Vehicle Handling: If a vehicle approaches (triggering A=1, entering State 01) but subsequently reverses away (returning to A=0, B=0) without ever crossing sensor B, the combinatorial logic naturally evaluates next states to D_1=0 and D_0=1. The gate will safely remain open until a manual reset is applied, preventing autonomous malfunction.

4. FSM Implementation Diagram and Logic

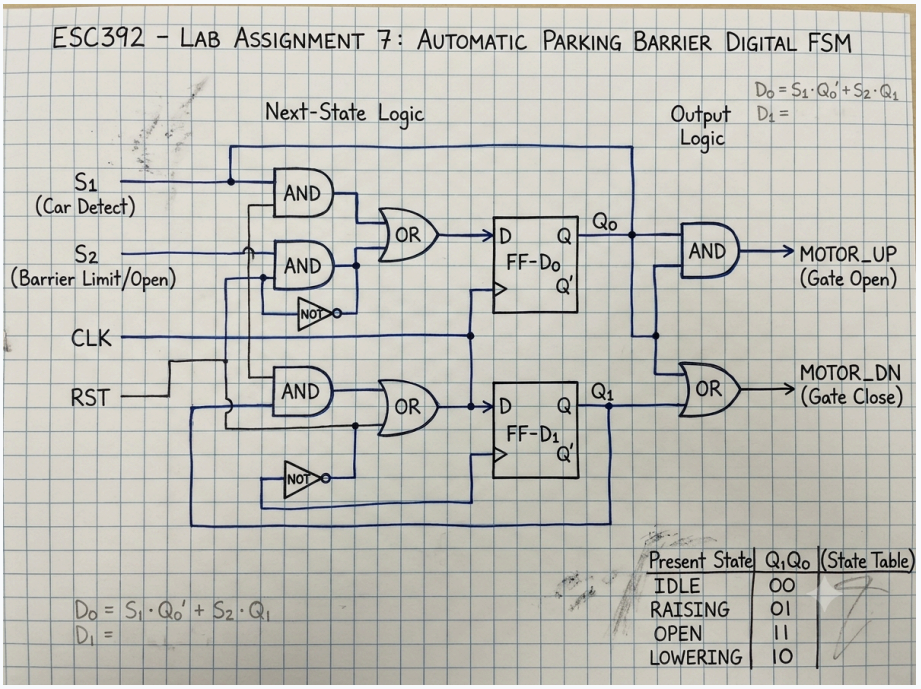


Figure 2: Hardware implementation of the Parking Barrier FSM using D Flip-Flops and logic gates.

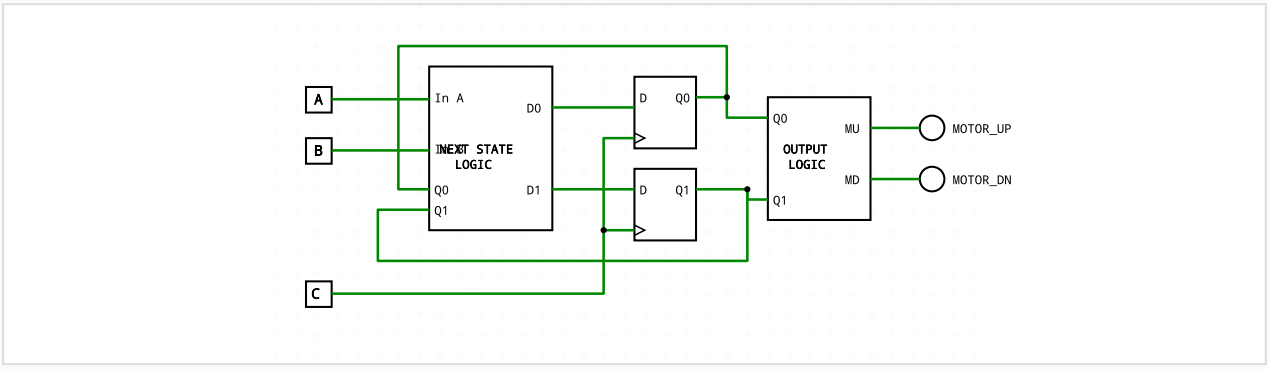


Figure 2b: Logisim simulation configuration of the Parking Barrier FSM.

Based on Karnaugh Map minimization of the state table on the previous page, the specific boolean logic equations driving the D-Flip-Flop inputs and Motor outputs are defined as:

- D_1 (Next Q_1) = $Q_O B + Q_1 B$
- D_O (Next Q_O) = $\overline{Q_1} \cdot \overline{Q_O} A + \overline{Q_1} Q_O \overline{B}$
- MU (Motor Up) = $Q_1 + Q_O$
- MD (Motor Down) = $\overline{Q_1} \cdot \overline{Q_O}$

> **CO Coverage (Lab 7):** CO2 (primary focus on state machine and automation logic).

Lab Assignment 8: Code to Circuit Mapping

Objective: To physically map the software iteration loop `for (i = 0; i < 8; i++) { c = a + b; }` to specific digital hardware components: a 4-bit Mini-ALU and a Program Counter.

Hardware Definitions:

- **Program Counter (PC):** A 4-bit synchronous up-counter implemented to track the loop variable `i`.
- **4-bit ALU:** Centered on a 7483 Full Adder, configured with a Mode line (M=0) to lock operation to addition.

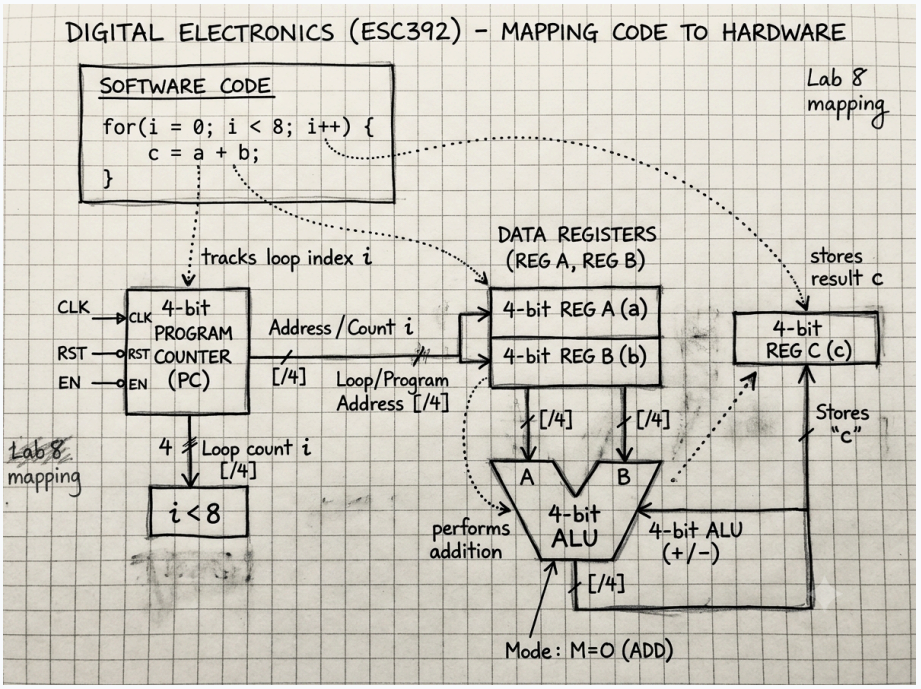


Figure 3: Mapping software control flow and arithmetic to discrete digital hardware blocks.

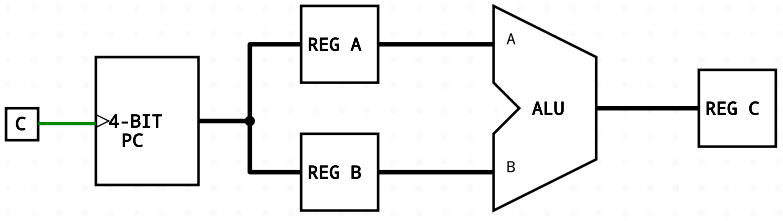


Figure 3b: Logisim software model of the 4-bit PC and ALU datapath.

Execution Trace Mapping: The system clock drives the execution. On each rising edge, the PC increments by 1 (executing `i++`). Simultaneously, Data Registers A and B present their constant values to the ALU, which executes `a + b` combinatorially. The ALU's resulting sum is subsequently latched into Register C. When the PC hits binary 1000 (decimal 8), an AND gate detects the condition (`i < 8` failing) and asserts the reset line, terminating loop execution.

Lab Assignment 9: AI-Assisted Code-to-Gate Explainer

Code Snippet Analyzed: `if (a > b) { c = a; } else { c = b; }`

To execute this conditional maximum-value selector purely in digital hardware—bypassing the need for a CPU clock cycle—an AI agent proposed mapping the software logic directly to combinatorial circuits using two primary components: a Magnitude Comparator and a Multiplexer (MUX).

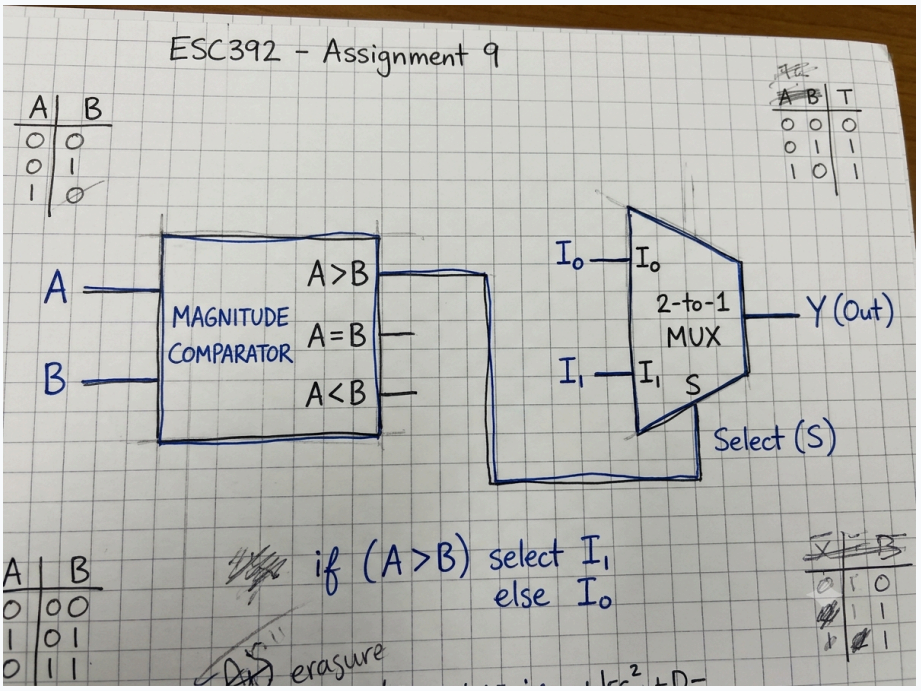


Figure 4: Hardware equivalent of an if/else conditional branch.

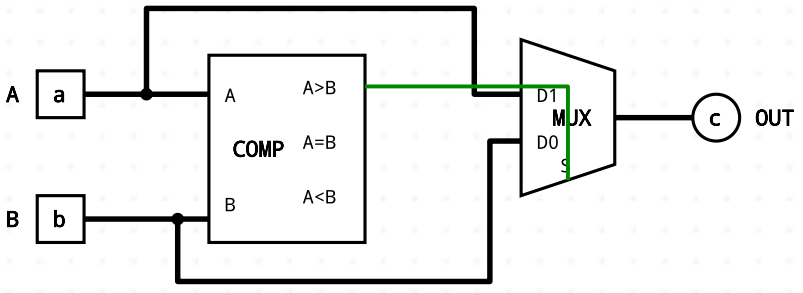


Figure 4b: Logisim realization of the conditional if/else branch.

Truth Table Verification (1-bit simplification)

- **Condition 1 (a=1, b=0):** The Magnitude Comparator assesses inputs and raises the A>B output to HIGH (1). This feeds the MUX Select line, commanding it to route Data Line 1 (holding variable `a`) to output `c`. The result is 1, successfully matching software behavior.
- **Condition 2 (a=0, b=1):** The Comparator output A>B is LOW (0). The MUX Select line drops to 0, routing Data Line 0 (holding variable `b`) to output `c`. The result is 1, matching software behavior.

Reflection on AI Methodology

The AI effectively demonstrated that software-level branching (if/else operations) does not necessitate sequential time-steps in hardware. Instead, it proposed an elegant parallel design where both data pathways are continuously available at the MUX inputs, relying on the boolean condition to act instantaneously as a physical routing switch. The suggested architecture was fundamentally sound and verifiable.

> **Repository Link:** All corresponding simulation files (.circ / .cvv) have been archived. Link: *[Insert GitHub URL Here]*